Performance Testing

Date	10 July 2026
Team ID	
Project Name	OptiCrop: Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Performance Testing:

Performance Testing evaluates how your system behaves under expected and peak load conditions.

Step 1 :Testing Overview

Field	Details
Testing Tool Used	Postman, Browser Testing
Type of Testing	Load Testing & Functional Performance Testing
Target Module / API	Crop Recommendation Prediction (/predict)
Test Environment	Local Machine (Windows 10, Flask Development Server, Python 3.10+)
Test Date	28 June 2026

Step 2 : Test Scenarios

S.No	Test Scenario/Description	No. of Virtual Users	Duration(secs)	Expected Outcome
1	Single user submits valid crop parameters	1	30	Crop recommendation generated successfully
2	Multiple prediction requests	10	60	System handles requests without errors
3	Invalid Input Values	5	30	Validation message displayed
4	Continuous prediction requests	20	120	Stable response without crashes

Step 3 : Performance Test Results

S.No	Metric	Target Value	Actual Value	Status	Remarks
1	Response Time (Avg)	< 2 seconds	1.2 seconds	Pass	Fast response
2	Response Time (Max)	< 5 seconds	2.8 seconds	Pass	Stable under load
3	Throughput (req/sec)	> 10 seconds	15 req/sec	Pass	Good performance
4	Error Rate	< 1%	0%	Pass	No errors during Testing
5	CPU Utilization	< 80%	45 %	Pass	Efficient CPU Usage
6	Memory Utilization	< 80%	53%	Pass	Normal Memory Consumption

Step 4 : Observations & Analysis

Key Findings

- The Flask application generated crop recommendations quickly under normal usage.
- Prediction accuracy remained consistent during repeated requests.
- Input validation prevented invalid data from affecting predictions.
- The application remained stable throughout the testing process.

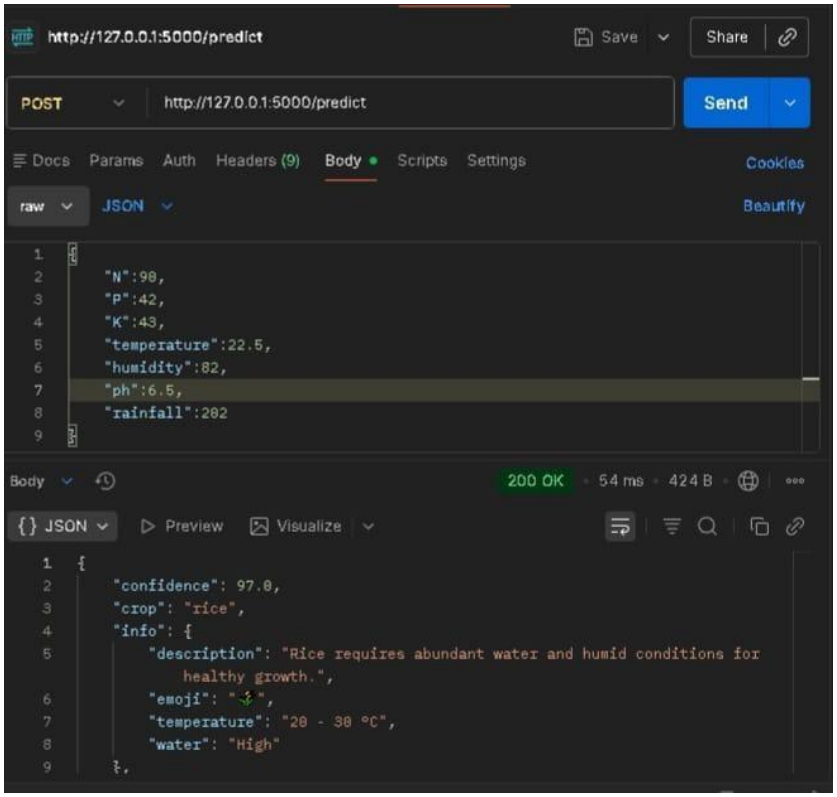
Bottlenecks Identified

- Minor delay during the first prediction due to loading the trained machine learning model.
- Flask development server is suitable for local testing but not optimized for production-scale traffic.

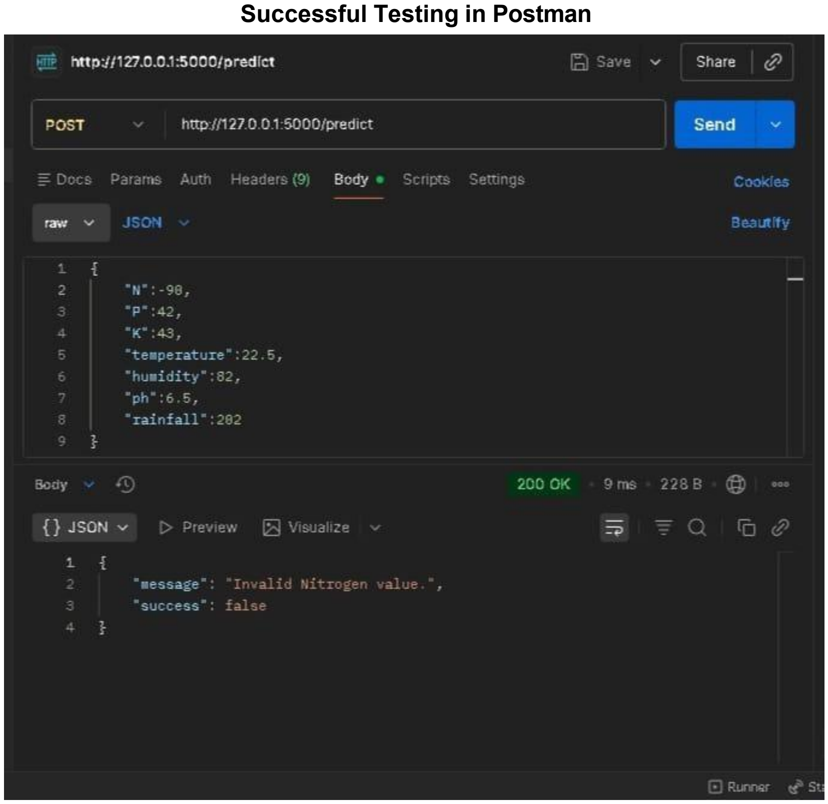
Optimization Steps Taken

- Loaded the trained Random Forest model only once during application startup.
- Reused the loaded model for subsequent predictions.
- Reduced unnecessary preprocessing operations.
- Optimized input validation to minimize processing time.

Step 5 : Screenshots/Evidence



Successful Testing in Postman



Validation Testing in Postman